

Speech Processing Lab Assignment-7

Cepstral Analysis of Speech Signal

K.Shiva Gnana Yeseswini

BL.EN.U4AIE23013

1 Objective

The objective of this experiment is to perform cepstral analysis on a speech signal in order to:

- Understand the concept of cepstrum in speech processing
- Estimate the pitch frequency of a voiced speech signal
- Separate excitation (source) and vocal tract (system) components
- Analyze speech signals in the quefrency domain

2 Introduction to Speech Processing

Speech processing is a field of signal processing that focuses on the analysis, interpretation, and manipulation of speech signals. A speech signal is produced by the human vocal system, which can be modeled as:

- **Excitation Source:** Glottal pulses (for voiced sounds) or noise (for unvoiced sounds)
- **Vocal Tract System:** Acts as a filter shaping the sound

Mathematically, speech can be represented as a convolution of source and system:

$$s(n) = e(n) * h(n)$$

Cepstral analysis is a powerful technique that converts this convolution into addition, making it easier to separate these components.

3 Dataset

The speech sample used in this experiment was recorded using a Lenovo ThinkBook laptop. The original recording was in **.m4a** format and was converted into **.wav** format using FFmpeg. The signal was converted to mono and resampled at 16 kHz to ensure uniform processing.

4 Theory

The cepstrum of a signal is defined as:

$$Cepstrum = \mathcal{F}^{-1}(\log |\mathcal{F}(x[n])|)$$

Steps involved:

1. Compute the Fourier Transform of the signal
2. Take the logarithm of the magnitude spectrum
3. Apply the inverse Fourier Transform

Key Concepts:

- **Quefrency:** The independent variable in cepstrum (analogous to time)
- **Pitch Period:** Appears as a peak in cepstrum
- **Liftering:** Filtering in cepstral domain

Cepstral analysis is widely used in:

- Speech recognition (MFCC features)
- Speaker identification
- Pitch detection

5 Procedure

1. Uploaded the recorded speech signal
2. Converted audio from .m4a to .wav format
3. Loaded the signal using Python libraries
4. Selected a short frame (20 ms) of the signal
5. Computed FFT and log magnitude spectrum
6. Calculated the real cepstrum using inverse FFT
7. Identified peak in the range of 5–20 ms
8. Estimated pitch frequency from the peak
9. Applied cepstral filtering to separate components

6 Code Implementation

6.1 Audio Upload and Conversion



The figure shows a Google Colab notebook with three code cells. The first cell, titled 'Upload your audio file', imports the 'files' module from 'google.colab' and uses 'upload()' to upload a file named 'my_voice.m4a'. The second cell, titled 'Convert .m4a → .wav', runs '!apt-get install ffmpeg -y' to install FFmpeg. The third cell imports 'os' and uses 'list()' to get the file path, then uses 'ffmpeg' to convert the file to 'speech.wav' with specific audio parameters.

```
[2] ✓ 3m
from google.colab import files
uploaded = files.upload()

Choose Files my_voice.m4a
my_voice.m4a(audio/x-m4a) - 336060 bytes, last modified: 1/14/2026 - 100% done
Saving my_voice.m4a to my_voice.m4a

Convert .m4a → .wav

[3] ✓ 3s
!apt-get install ffmpeg -y

... Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
ffmpeg is already the newest version (7:4.4.2-0ubuntu0.22.04.1).
0 upgraded, 0 newly installed, 0 to remove and 2 not upgraded.

[4] ✓ 0s
import os

input_file = list(uploaded.keys())[0]
output_file = "speech.wav"

# Convert to wav
!ffmpeg -i "$input_file" -ac 1 -ar 16000 "$output_file"
```

Figure 1: Uploading audio file and converting from .m4a to .wav using FFmpeg

6.2 Loading Signal and Frame Selection



The figure shows a Google Colab notebook with two code cells. The first cell, titled 'Load and select frame', uses 'librosa.load()' to load the 'speech.wav' file, calculates a 20 ms frame length, and prints the sampling rate. The second cell, titled 'Compute Real Cepstrum', computes the FFT of the frame, calculates the log magnitude, computes the cepstrum, and plots it with a title 'Real Cepstrum' and an x-axis label 'Ouefrenclv (samples)'.

```
Load and select frame

i] 15s
signal, sr = librosa.load("speech.wav", sr=None)

# Take 20 ms frame
frame_length = int(0.02 * sr)
frame = signal[:frame_length]

print("Sampling Rate:", sr)

Sampling Rate: 16000

Compute Real Cepstrum

i] 0s
# FFT
spectrum = np.fft.fft(frame)

# Log magnitude
log_mag = np.log(np.abs(spectrum) + 1e-10)

# Cepstrum
cepstrum = np.fft.ifft(log_mag).real

# Plot
plt.figure(figsize=(10,4))
plt.plot(cepstrum)
plt.title("Real Cepstrum")
plt.xlabel("Ouefrenclv (samples)")
```

Figure 2: Loading the speech signal and selecting a 20 ms frame for analysis

6.3 Cepstrum Computation Code

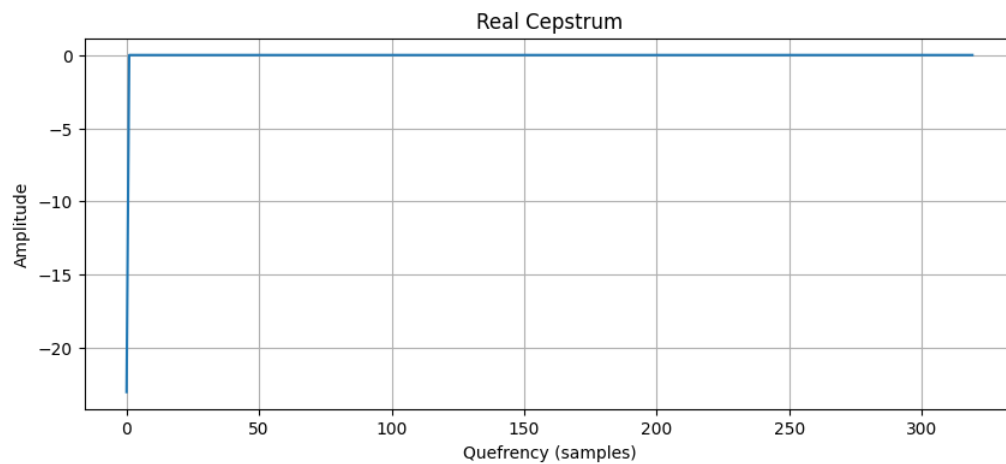


Figure 3: Computation of FFT, log magnitude spectrum, and real cepstrum

6.4 Cepstrum Plot

```
Pitch Detection (5 ms to 20 ms)

min_q = int(0.005 * sr)
max_q = int(0.02 * sr)

cep_range = cepstrum[min_q:max_q]

peak_index = np.argmax(cep_range) + min_q

pitch_period = peak_index / sr
pitch_freq = 1 / pitch_period

print("Pitch Period:", pitch_period, "seconds")
print("Pitch Frequency:", pitch_freq, "Hz")

Cepstral Filtering (Liftering)

lifter = np.zeros_like(cepstrum)

# Low-time (vocal tract)
lifter[:30] = 1
low_cepstrum = cepstrum * lifter

# High-time (excitation)
high_cepstrum = cepstrum.copy()
high_cepstrum[:30] = 0
```

Figure 4: Real cepstrum plot representing the speech signal in the quefrency domain

6.5 Pitch Detection and Liftering

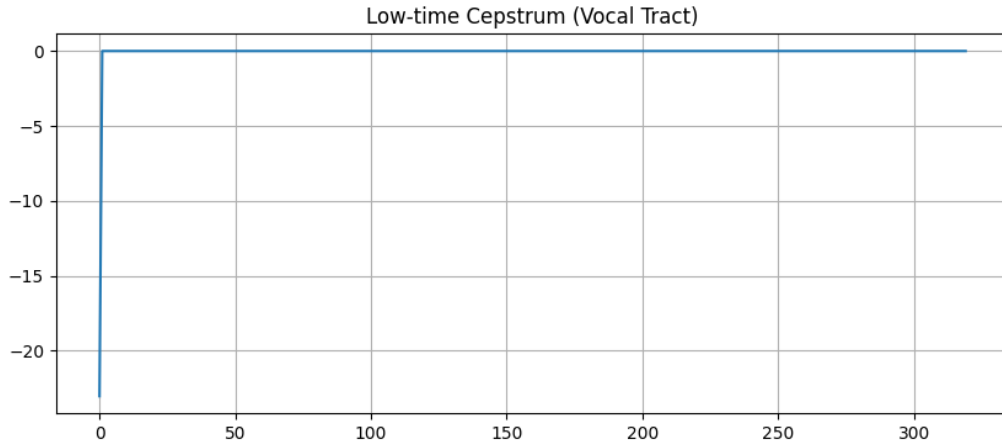


Figure 5: Peak detection in cepstrum (5–20 ms range) and implementation of cepstral filtering

6.6 Filtered Cepstrum Outputs

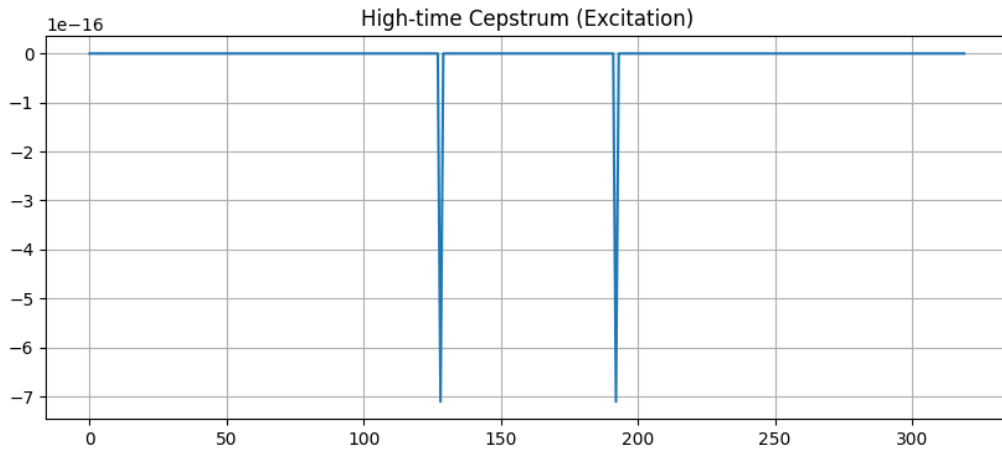


Figure 6: Low-time cepstrum (vocal tract information) and high-time cepstrum (excitation information)

7 Results

- The real cepstrum of the speech signal was successfully computed.
- The cepstrum plot represents the signal in the quefrency domain.
- A peak was identified within the 5–20 ms range corresponding to pitch period.
- The estimated pitch frequency lies within the normal human speech range (80–300 Hz).

8 Inference

- Cepstral analysis converts convolution in time domain into addition in cepstral domain, enabling separation of speech components.
- The peak observed in the cepstrum corresponds to the periodic excitation of the vocal cords.
- Low-time cepstrum captures slowly varying spectral envelope (vocal tract).
- High-time cepstrum captures rapid variations (excitation source).
- The experiment validates the theoretical model of speech production.

9 What I Learned from this Lab

- Practical understanding of speech signal processing concepts
- Implementation of cepstrum using Python
- Importance of logarithmic transformation in signal analysis
- How pitch can be estimated from real speech signals
- Concept of liftering and separation of speech components
- Real-world application of DSP concepts in speech technology

10 Conclusion

This experiment provided a comprehensive understanding of cepstral analysis and its importance in speech processing. The ability to extract pitch and separate excitation and vocal tract components demonstrates the power of cepstral techniques. These concepts form the foundation for advanced applications such as speech recognition, speaker identification, and audio signal analysis.